

Pairing Claude Code with Local Models

 kdnuggets.com/pairing-claude-code-with-local-models

Shittu Olumide

Introduction

Agentic coding sessions are expensive. A single Claude Code session — reading files, writing code, running tests, iterating — can burn 10–50x more tokens than a plain chat conversation. At scale, that adds up fast. Add rate limits that can interrupt a long-running workflow mid-session, and the dependency on a third-party API that can change pricing, enforce stricter policies, or go down at any point, and the case for local inference becomes straightforward.

Local models in 2026 are good enough. For the tasks Claude Code handles daily — code completion, refactoring, debugging, codebase explanation — a well-chosen quantized model running locally covers the vast majority of real use cases at zero per-token cost and with no rate limits. This article covers three inference backends ([Ollama](#), [LM Studio](#), and [llama.cpp](#)), the exact environment variables and configuration files to wire each one to Claude Code, a curated table of models worth running, and the troubleshooting fixes for the issues you will actually hit.

How Claude Code Connects to Any Local Model

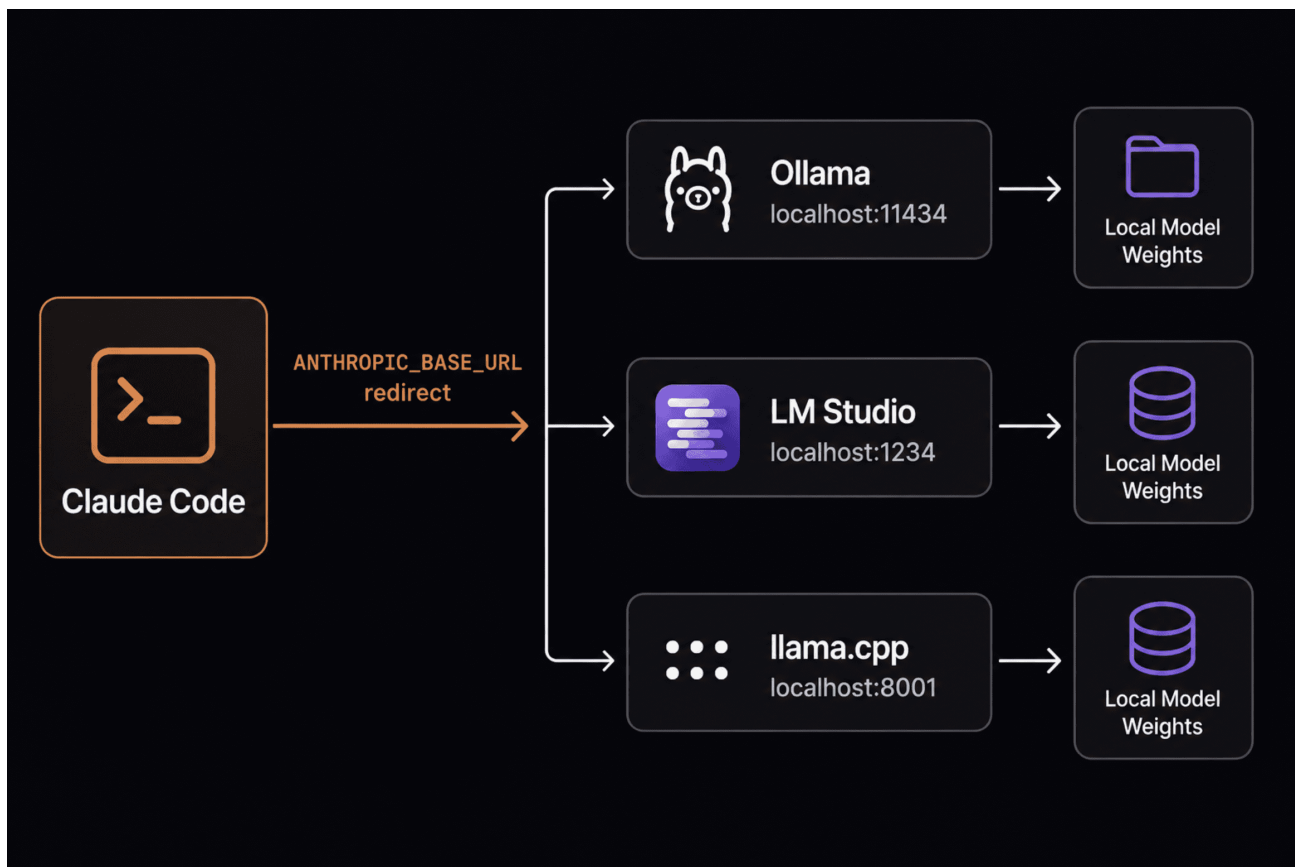
The mechanism is simpler than most guides make it look. Claude Code sends requests in the Anthropic Messages API format. By default those requests go to Anthropic's servers. Setting `ANTHROPIC_BASE_URL` redirects them to any server that speaks the same format, which now includes Ollama, LM Studio, and llama.cpp natively.

According to the official Claude Code environment variables documentation, the variables that matter for this setup are:

- `ANTHROPIC_BASE_URL`: redirects all API calls from Anthropic's servers to whatever URL you set. Set this to your local inference server address.
- `ANTHROPIC_API_KEY`: the API key sent in the request header. Local servers typically ignore authentication, so this is usually set to a placeholder string like **"local"** or **"ollama."**
- `ANTHROPIC_AUTH_TOKEN`: an alternative auth header. Some local servers check for this instead of the API key. Set it to the same placeholder.

`ANTHROPIC_DEFAULT_SONNET_MODEL`, `ANTHROPIC_DEFAULT_HAIKU_MODEL`, and `ANTHROPIC_DEFAULT_OPUS_MODEL`: Claude Code internally requests different model tiers depending on the task. These three variables map each tier to your local model's name. Without them, Claude Code sends requests for `claude-sonnet-4-20250514` to your local server, which will reject the request because no such model exists locally.

In January 2026, Ollama added native support for the Anthropic Messages API, which was the technical change that made this workflow practical without translation proxies. LM Studio added a native `/v1/messages` endpoint in version 0.4.1. llama.cpp has had direct Anthropic API support for longer. All three now speak Claude Code's native protocol.



A clean architecture diagram showing Claude Code, Ollama, LM Studio, and llama.cpp | Image by Author

Backend 1: Ollama

Ollama is the right starting point. It handles all the complexity of model management — downloading weights, quantization, GPU and CPU allocation, and serving — behind a simple command-line interface (CLI). One command to install, one command to pull a

model, a few environment variables to configure. It runs as a background service after install, so there is no manual server start required.

Prerequisites

- macOS, Linux, or Windows (WSL2 recommended on Windows)
- At least 16 GB RAM for practical use (32 GB recommended)
- GPU with 8+ GB VRAM for GPU inference, or CPU-only with enough RAM
- Ollama v0.14.0 or later required for Anthropic Messages API support

Install Ollama:

```
# macOS and Linux -- one command install
curl -fsSL https://ollama.com/install.sh | sh

# Verify the version -- must be 0.14.0+ for Claude Code compatibility
ollama version
# Expected: ollama version is 0.14.x or higher

# Windows: download the installer from https://ollama.com
# Native Windows support has improved significantly in recent releases
```

After installation, Ollama starts automatically as a background service on port **11434**. You can verify it is running:

```
# Check the Ollama server is live
curl http://localhost:11434

# Expected response:
# Ollama is running
```

Pull a coding model:

```
# GLM-4.7-Flash -- recommended starting point
# Strong tool calling, 128K context, fits on 8 GB VRAM
# Apache 2.0 license
ollama pull glm-4.7-flash:latest

# Qwen3-Coder -- strong code generation and instruction following
# Requires 20+ GB VRAM for the full model
ollama pull qwen3-coder

# Devstral-Small -- specifically designed for agentic coding workflows
# Community-tested for Claude Code compatibility
# 24B, requires 16+ GB VRAM
ollama pull devstral-small-2:24b

# Verify the model is downloaded and ready
ollama list
# Shows all pulled models with their sizes and modification dates
```

// Configuring Claude Code to Use Ollama

Option 1: Shell export (current terminal session only)

```
# Redirect Claude Code to your local Ollama server
export ANTHROPIC_BASE_URL="http://localhost:11434"

# Local servers do not require real authentication
# Set these to any non-empty string -- Ollama ignores the value
export ANTHROPIC_API_KEY="ollama"
export ANTHROPIC_AUTH_TOKEN="ollama"

# Map Claude Code's model tier requests to your local model name
# Claude Code internally requests sonnet/haiku/opus -- these variables
# translate those tier names to whatever model you have pulled locally
export ANTHROPIC_DEFAULT_SONNET_MODEL="glm-4.7-flash:latest"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="glm-4.7-flash:latest"
export ANTHROPIC_DEFAULT_OPUS_MODEL="glm-4.7-flash:latest"

# Launch Claude Code -- it will now use Ollama instead of the Anthropic API
claude
```

Option 2: ~/.claude/settings.json (permanent, applies to all sessions)

This approach survives terminal restarts and applies every time you launch Claude Code. Claude Code reads environment variables from settings.json at startup so they take effect no matter how claude was launched.

Create or edit ~/.claude/settings.json:

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:11434",
    "ANTHROPIC_API_KEY": "ollama",
    "ANTHROPIC_AUTH_TOKEN": "ollama",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "glm-4.7-flash:latest",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "glm-4.7-flash:latest",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "glm-4.7-flash:latest"
  }
}
```

Option 3: .env file in project directory (per-project override)

If you want a specific project to use a different model while keeping your global settings on the Anthropic API:

```
# .env in your project root -- loaded automatically by Claude Code
ANTHROPIC_BASE_URL=http://localhost:11434
ANTHROPIC_API_KEY=ollama
ANTHROPIC_AUTH_TOKEN=ollama
ANTHROPIC_DEFAULT_SONNET_MODEL=qwen3-coder
```

```
ANTHROPIC_DEFAULT_HAIKU_MODEL=qwen3-coder
ANTHROPIC_DEFAULT_OPUS_MODEL=qwen3-coder
```

Verify the connection:

```
# Launch Claude Code with a simple test
claude

# Inside Claude Code, run a basic prompt:
# > What model are you running?
# A local model should respond without making any Anthropic API calls.

# To confirm no external calls are being made, run with verbose logging:
claude --verbose

# Look for lines showing requests going to localhost:11434
# rather than api.anthropic.com
```

Full working sequence from scratch:

```
curl -fsSL https://ollama.com/install.sh | sh          # 1. Install Ollama
ollama pull glm-4.7-flash:latest                       # 2. Pull model (~4 GB)
export ANTHROPIC_BASE_URL="http://localhost:11434"     # 3. Redirect Claude Code
export ANTHROPIC_API_KEY="ollama"                     # 4. Set placeholder auth
export ANTHROPIC_AUTH_TOKEN="ollama"
export ANTHROPIC_DEFAULT_SONNET_MODEL="glm-4.7-flash:latest"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="glm-4.7-flash:latest"
export ANTHROPIC_DEFAULT_OPUS_MODEL="glm-4.7-flash:latest"
claude                                                  # 5. Launch
```

Backend 2: LM Studio

LM Studio is the right choice if you want a graphical interface for browsing and managing models rather than working entirely in the terminal. Since version 0.4.1, it includes a native Anthropic-compatible **/v1/messages** endpoint — the same path Claude Code expects — so no translation layer or proxy is needed.

Prerequisites:

- macOS, Windows, or Linux
- GPU with 6+ GB VRAM recommended (CPU-only is possible but slow)
- Download from lmstudio.ai or use the CLI installer for headless servers

Install and configure LM Studio:

```
# On a server or VM without a GUI -- CLI installer
curl -fsSL https://releases.lmstudio.ai/cli/install.sh | bash

# Or download the desktop app from https://lmstudio.ai for GUI use
```

GUI setup steps:

1. Open LM Studio and search for a coding model (search "qwen coder" or "devstral").
2. Download the model. LM Studio handles quantization selection automatically.
3. Go to the **Local Server** tab (the <> icon in the left sidebar).
4. Set the context size. LM Studio recommends starting with at least 25,000 tokens and increasing for better results.
5. Click **Start Server**.
6. Note the port (default: 1234) and copy the model name exactly as shown.

Note: Copy the model identifier exactly. LM Studio displays the exact string you need to pass to ANTHROPIC_DEFAULT_SONNET_MODEL. A mismatch here is the most common failure mode.

Configure Claude Code:

```
# Set the base URL to LM Studio's local server
export ANTHROPIC_BASE_URL="http://localhost:1234"
export ANTHROPIC_API_KEY="lm-studio"
export ANTHROPIC_AUTH_TOKEN="lm-studio"

# Replace the model name with what LM Studio shows for your loaded model
# Copy it exactly -- including any version suffix or quantization tag
export ANTHROPIC_DEFAULT_SONNET_MODEL="qwen2.5-coder-32b-instruct"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="qwen2.5-coder-32b-instruct"
export ANTHROPIC_DEFAULT_OPUS_MODEL="qwen2.5-coder-32b-instruct"
```

Or persistently in ~/.claude/settings.json:

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:1234",
    "ANTHROPIC_API_KEY": "lm-studio",
    "ANTHROPIC_AUTH_TOKEN": "lm-studio",
    "ANTHROPIC_DEFAULT_SONNET_MODEL": "qwen2.5-coder-32b-instruct",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "qwen2.5-coder-32b-instruct",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "qwen2.5-coder-32b-instruct"
  }
}
```

How to run:

```
# 1. Start the LM Studio server from the GUI (Local Server tab > Start Server)
# 2. Set environment variables
export ANTHROPIC_BASE_URL="http://localhost:1234"
export ANTHROPIC_API_KEY="lm-studio"
export ANTHROPIC_AUTH_TOKEN="lm-studio"
export ANTHROPIC_DEFAULT_SONNET_MODEL="your-model-name-here"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="your-model-name-here"
export ANTHROPIC_DEFAULT_OPUS_MODEL="your-model-name-here"
# 3. Launch
claude
```

Backend 3: llama.cpp

llama.cpp is the right choice when you need direct control over inference parameters — quantization type, KV cache configuration, batch size, thread count — or when you are running on a server and want the lowest overhead. It has native Anthropic Messages API support, so no proxy or translation layer is needed.

Prerequisites:

- A GGUF-format model file (download from Hugging Face; search for "GGUF" versions of any model)
- CUDA-capable GPU for GPU inference, or CPU-only for slower inference
- CMake and a C++ compiler for source builds (on Linux/CUDA, source is recommended)

Install llama.cpp:

```
# macOS -- Homebrew is simplest
brew install llama.cpp

# Linux with CUDA -- build from source for best GPU performance
git clone https://github.com/ggml-org/llama.cpp
cd llama.cpp
cmake -B build -DGGML_CUDA=ON          # Enable CUDA acceleration
cmake --build build --config Release    # Build
# Binaries in ./build/bin/

# Linux CPU-only build
cmake -B build
cmake --build build --config Release

# Windows -- pre-built binaries available at:
# https://github.com/ggml-org/llama.cpp/releases
# Download the CUDA or CPU variant matching your hardware
```

Download a GGUF model:

```
# Install the Hugging Face CLI if you do not have it
pip install huggingface-hub

# Download GLM-4.7-Flash in Q4_K_XL quantization (~4.5 GB)
# This quantization offers a good size/quality balance for coding
huggingface-cli download unsloth/GLM-4.7-Flash-GGUF \
  GLM-4.7-Flash-UD-Q4_K_XL.gguf \
  --local-dir ./models/

# Or download Qwen3-Coder in Q4 quantization (~15 GB for 32B)
huggingface-cli download Qwen/Qwen3-Coder-32B-Instruct-GGUF \
  qwen3-coder-32b-instruct-q4_k_m.gguf \
  --local-dir ./models/
```

Start the llama.cpp server:

```
# Start llama-server with Anthropic API support and a 128K context window
llama-server \
  --model ./models/GLM-4.7-Flash-UD-Q4_K_XL.gguf \
  --alias "glm-4.7-flash" \           # This name goes in ANTHROPIC_DEFAULT_SONNET_MODEL
  --port 8001 \
  --ctx-size 131072 \               # 128K context -- important for large codebases
  --flash-attn \                     # Memory-efficient attention, improves speed
  --n-gpu-layers 99                 # Offload all layers to GPU; remove for CPU-only

# For CPU-only inference (no GPU):
llama-server \
  --model ./models/GLM-4.7-Flash-UD-Q4_K_XL.gguf \
  --alias "glm-4.7-flash" \
  --port 8001 \
  --ctx-size 32768 \               # Reduce context size on CPU to keep memory manageable
  --threads 8                      # Match your CPU core count
```

Key flags explained:

- `--alias`: the model name string Claude Code will send in requests. Set `ANTHROPIC_DEFAULT_SONNET_MODEL` to match this exactly.
- `--ctx-size`: context window in tokens. **131072 = 128K**. Larger is better for codebase analysis but uses more VRAM. Reduce if you get out-of-memory errors.
- `--flash-attn`: Flash Attention reduces peak VRAM by processing attention in smaller blocks. Enable it whenever your build supports it.
- `--n-gpu-layers 99`: offloads all transformer layers to the GPU. The server automatically uses fewer layers if VRAM is tight.

Configure Claude Code:

```
export ANTHROPIC_BASE_URL="http://localhost:8001"
export ANTHROPIC_API_KEY="llama-cpp"
export ANTHROPIC_AUTH_TOKEN="llama-cpp"

# Must match the --alias you passed to llama-server exactly
export ANTHROPIC_DEFAULT_SONNET_MODEL="glm-4.7-flash"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="glm-4.7-flash"
export ANTHROPIC_DEFAULT_OPUS_MODEL="glm-4.7-flash"
```

How to run:

```
# Terminal 1: start the llama.cpp server
llama-server \
  --model ./models/GLM-4.7-Flash-UD-Q4_K_XL.gguf \
  --alias "glm-4.7-flash" \
  --port 8001 \
  --ctx-size 131072 \
  --flash-attn \
  --n-gpu-layers 99
```



```
# Terminal 2: configure and launch Claude Code
export ANTHROPIC_BASE_URL="http://localhost:8001"
export ANTHROPIC_API_KEY="llama-cpp"
export ANTHROPIC_AUTH_TOKEN="llama-cpp"
export ANTHROPIC_DEFAULT_SONNET_MODEL="glm-4.7-flash"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="glm-4.7-flash"
export ANTHROPIC_DEFAULT_OPUS_MODEL="glm-4.7-flash"
claude
```

The Complete settings.json

Environment variable exports last only as long as the terminal session. For a durable configuration, use `~/.claude/settings.json`. Claude Code reads variables from this file at startup so they apply no matter how Claude was launched — from the terminal, from a VS Code task, or from a script.

Here is a production-ready `settings.json` with all variables explained:

```
{
  "env": {
    "ANTHROPIC_BASE_URL": "http://localhost:11434",

    "ANTHROPIC_API_KEY": "ollama",
    "ANTHROPIC_AUTH_TOKEN": "ollama",

    "ANTHROPIC_DEFAULT_SONNET_MODEL": "glm-4.7-flash:latest",
    "ANTHROPIC_DEFAULT_HAIKU_MODEL": "glm-4.7-flash:latest",
    "ANTHROPIC_DEFAULT_OPUS_MODEL": "glm-4.7-flash:latest",

    "CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS": "1"
  }
}
```

Why `CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS: "1"` matters:

When using Claude Code through non-Anthropic backends, Claude Code adds Anthropic-specific experimental beta flags to request headers — flags that third-party and local servers do not recognize. This causes `Error: Unexpected value(s) for the anthropic-beta header` on most local inference servers. Setting this variable to `"1"` strips those headers before the request goes out, which eliminates the error without affecting any core Claude Code functionality.

Switching between backends:

If you work with multiple backends — Ollama for daily use, the Anthropic API for complex tasks — the cleanest approach is maintaining separate shell scripts rather than editing `settings.json` back and forth:

```
# use-local.sh -- switch to Ollama
export ANTHROPIC_BASE_URL="http://localhost:11434"
export ANTHROPIC_API_KEY="ollama"
export ANTHROPIC_AUTH_TOKEN="ollama"
export ANTHROPIC_DEFAULT_SONNET_MODEL="glm-4.7-flash:latest"
export ANTHROPIC_DEFAULT_HAIKU_MODEL="glm-4.7-flash:latest"
export ANTHROPIC_DEFAULT_OPUS_MODEL="glm-4.7-flash:latest"
echo "Claude Code → local Ollama (glm-4.7-flash)"

# use-anthropic.sh -- switch back to the Anthropic API
unset ANTHROPIC_BASE_URL
unset ANTHROPIC_AUTH_TOKEN
unset ANTHROPIC_DEFAULT_SONNET_MODEL
unset ANTHROPIC_DEFAULT_HAIKU_MODEL
unset ANTHROPIC_DEFAULT_OPUS_MODEL
# ANTHROPIC_API_KEY should already be set to your real key in your rc file
echo "Claude Code → Anthropic API"
```

Source either script in your current session:

```
source ./use-local.sh
claude

# When you need the real API for a complex task:
source ./use-anthropic.sh
claude
```

Best Local Models for Claude Code in 2026

Hardware is the main constraint. For Claude Code with local models to be genuinely usable for coding tasks rather than just a demo, aim for 32 GB of RAM — Apple Silicon unified memory or PC RAM. 16 GB is viable with smaller quantized models and CPU offload, but generation speed will be noticeably slower on multi-step agentic tasks.

Model	VRAM Needed	Context	Strengths	License	Pull Command
glm-4.7-flash	8 GB	128K	Tool calling, fast, low VRAM	Apache 2.0	ollama pull glm-4.7-flash
devstral-small-2:24b	16 GB	32K	Agentic coding workflows	Apache 2.0	ollama pull devstral-small- 2:24b
qwen3-coder	20 GB	128K	Code generation, instructions	Apache 2.0	ollama pull qwen3-coder

Model	VRAM Needed	Context	Strengths	License	Pull Command
gwen3.5:27b	20 GB	256K	Strong all-round, huge context	Apache 2.0	ollama pull gwen3.5:27b
gemma4:26b	20 GB	256K	Reasoning, 77% coding bench	Gemma License	ollama pull gemma4:26b

Troubleshooting Common Issues

1. **Connection refused when launching Claude Code:** The inference server is not running. This is the most common issue and the easiest to diagnose.

```
# Check if Ollama is running
curl http://localhost:11434
# Expected: "Ollama is running"
```

```
# Check if LM Studio server is running
curl http://localhost:1234/v1/models
# Should return a JSON list of loaded models
```

```
# Check if llama-server is running
curl http://localhost:8001/health
# Should return {"status":"ok"}
```

```
# If not running -- start the server first, then launch Claude Code
ollama serve          # Ollama
# LM Studio: use the GUI Local Server tab
# llama.cpp: run the llama-server command from the Backend 3 section
```

2. **Model not found or unknown model error:** The model name in your ANTHROPIC_DEFAULT_SONNET_MODEL does not match what the server knows.

```
# List all models Ollama has available
ollama list
```

```
# The model name in ANTHROPIC_DEFAULT_SONNET_MODEL must match EXACTLY
# including the tag -- "glm-4.7-flash:latest" not "glm-4.7-flash"
```

```
# Verify with a direct API call to confirm what the server sees
curl http://localhost:11434/v1/models
```

3. **Tool calls failing or returning errors:** For streaming tool calls, which Claude Code uses when executing functions or scripts, Ollama version 0.14.3-rc1 or later is required. Earlier versions in the 0.14.x series had incomplete streaming tool call support.

```
# Check your Ollama version
ollama version

# If below 0.14.3, update Ollama
curl -fsSL https://ollama.com/install.sh | sh
```

4. anthropic-beta header error:

You will see: Error: Unexpected value(s) for the anthropic-beta header. This happens because Claude Code adds Anthropic-specific experimental beta flags that local servers do not recognize. Fix it by adding this to your settings.json env block:

```
"CLAUDE_CODE_DISABLE_EXPERIMENTAL_BETAS": "1"
```

5. **Reverting to the Anthropic API:**

```
# Shell session -- unset the redirect variables
unset ANTHROPIC_BASE_URL
unset ANTHROPIC_AUTH_TOKEN
unset ANTHROPIC_DEFAULT_SONNET_MODEL
unset ANTHROPIC_DEFAULT_HAIKU_MODEL
unset ANTHROPIC_DEFAULT_OPUS_MODEL

# Then make sure your real API key is set
echo $ANTHROPIC_API_KEY
# Should show your sk-ant-... key, not a placeholder

# If you used settings.json -- remove or comment out the env block
# and restart Claude Code
```

6. **Slow generation speed:** For agentic Claude Code tasks, generation speed matters because each tool call is a round trip. If speed is inadequate:
- Switch to a smaller or more aggressively quantized model (Q4_K_M instead of Q8).
 - Enable `--flash-attn` in llama.cpp if not already set.
 - Reduce context size (`--ctx-size`); larger contexts are slower to prefill.
 - On Ollama, set `OLLAMA_NUM_GPU_LAYERS=99` in your environment to force maximum GPU offload.

Conclusion

What used to require fragile adapters and hacks is now a five-step process. Install the inference backend, pull a model, set three environment variables, and Claude Code routes

to your local machine instead of Anthropic's API. The configuration takes under five minutes once you have the model downloaded.

The practical result is a coding assistant that costs nothing to run after setup, has no rate limits, keeps your code entirely on your machine, and covers the vast majority of real coding use cases at quality levels that were not available in local models a year ago. Start with Ollama and `glm-4.7-flash` — it has the lowest hardware requirement, the most consistent tool-calling support, and the fastest path to a working setup. Once that is running, scale up the model based on your hardware and the quality level you actually need.